Architectural Design Document - Plant Food Safety Text Classification

1. Project Overview

This project implements a progressive deep learning pipeline to classify natural language text descriptions of plants and fungi into three categories: `edible`, `poisonous`, and `food_poisoning`.

The Core Challenge: The dataset features a severe class imbalance (~3.8% `food_poisoning` samples vs. ~60% `edible` samples).

The Solution Strategy: The pipeline utilizes `RandomOverSampler` to synthetically balance the classes during training. Furthermore, it employs a **Sequential Embedding Transfer** strategy. Instead of training models in isolation, each model initializes its embedding layer using the refined weights of the previous model, creating a "curriculum learning" effect where the semantic map becomes increasingly sophisticated.

---

2. Model Architectures Detail

Model 1: Artificial Neural Network (ANN)

Role: The Baseline & Seed Embedding Generator.

Architecture Flow: `Embedding → GlobalAveragePooling1D → Dense (512) → Dense (256) → Dense (128) → Softmax`

- **Documentation:** This is a pure Feed-Forward Network designed to establish the foundational word embeddings. It completely ignores word order. Instead, the `GlobalAveragePooling1D` layer averages the high-dimensional vectors of every token in the 300-word sequence into a single "document vector."
- **Intuition:** It acts as a "bag-of-words" model. It learns that words like "fatal" and "alkaloid" push the mathematical vector toward the `poisonous` class, while "sweet" pushes it toward `edible`, without worrying about grammar or sentence structure.

Model 2: Convolutional Neural Network (TextCNN)

Role: Local Feature & N-Gram Detector.

Architecture Flow: `ANN Embedding → Parallel Conv1D Branches (k=2,3,4,5) → GlobalMaxPooling1D → Concatenate → Dense (256) → Dense (128) → Softmax`

- **Documentation:** This model adapts image-processing convolutions for text. It inherits the ANN's embedding map and passes it through four parallel 1D Convolutional branches with varying window sizes (2, 3, 4, and 5 words).
- **Intuition:** By sliding these windows across the text, the CNN acts as an n-gram detector. It identifies specific local phrases or idioms (e.g., a 3-word window perfectly captures "causes severe vomiting") regardless of where they appear in the paragraph. `GlobalMaxPooling1D` extracts the most dominant feature from each window size before classification.

Model 3: Transformer Encoder

Role: Global Context & Self-Attention.

Architecture Flow: `CNN Embedding → Multi-Head Attention (4 heads) → Add & Norm → Feed-Forward → Add & Norm → GlobalAveragePooling1D → Dense (128) → Softmax`

- **Documentation:** This pure Transformer block abandons sequential processing entirely. It inherits the CNN's n-gram-aware embedding. The `MultiHeadAttention` layer allows every single word in the 300-word sequence to mathematically attend to every other word simultaneously.
- **Intuition:** It resolves long-range dependencies and disambiguates vocabulary. For example, the word "leaves" means something entirely different if the word "tea" is at the beginning of the sentence versus if the word "toxic" is 50 words away. The Transformer maps this global context natively.

Model 4: Bidirectional LSTM with Additive Attention (BiLSTM)

Role: Sequential Memory & Temporal Context.

Architecture Flow: `Transformer Embedding → SpatialDropout1D → BiLSTM (128) → BiLSTM (64) → Custom Additive Attention → Dense (256) → Dense (128) → Softmax`

- **Documentation:** This Recurrent Neural Network processes the text sequentially, reading left-to-right and right-to-left simultaneously. The custom `AdditiveAttention` layer sits on top of the recurrent states to dynamically weigh the importance of specific time-steps (words). `SpatialDropout1D` is utilized to drop entire 1D feature maps, forcing the model to not rely on specific token representations.
- **Intuition:** While the Transformer looks at everything at once, the LSTM "reads" the text like a human, maintaining a hidden state memory of the sentence flow. The Additive Attention layer acts as a highlighter, allowing the model to mathematically focus on the 5 or 6 critical words in a 300-word paragraph that determine toxicity.

Model 5: Bidirectional GRU with Multi-Head Attention (BiGRU)

Role: Optimized Recurrence with Hybrid Attention.

Architecture Flow: `LSTM Embedding → SpatialDropout1D → BiGRU (128) → BiGRU (64) → Multi-Head Attention (2 heads) → Residual Add → Dense (256) → Dense (128) → Softmax`

- **Documentation:** Gated Recurrent Units (GRUs) provide a computationally lighter alternative to LSTMs by combining the forget and input gates. This model creates a hybrid architecture by placing a `MultiHeadAttention` layer *after* the recurrent processing, fusing sequential reading with global self-attention.
- **Intuition:** This model represents the absolute pinnacle of the embedding transfer chain. By the time the text reaches the classification head, the mathematical representations of the words have been informed by average pooling, local convolutions, pure self-attention, and sequential memory gating.

---

3. Meta-Learning & Ensembles

To maximize predictive accuracy and resolve individual architectural biases, the pipeline utilizes three distinct ensemble strategies.

3.1. Unified Neural Ensemble (The Super-Graph)

Instead of averaging outputs post-training, this is a distinct, monolithic Keras model.

- **Architecture:** It utilizes a single, shared embedding layer (seeded by the final BiGRU). The input branches out into five parallel sub-networks: an ANN branch, a CNN branch, a Transformer branch, an LSTM branch, and a GRU branch.
- **Fusion:** The high-level mathematical feature vectors from all five branches are concatenated together (`layers.Concatenate`). This massive fused vector is passed through a Meta-Classifier (Dense 512 → Batch Normalization → Dense 256).
- **Intuition:** This allows the final dense layers to make a decision by simultaneously looking at local n-grams (from the CNN branch), sequence memory (from the RNN branches), and global context (from the Transformer branch).

3.2. Soft-Voting Meta-Learner

- **Method:** A post-training statistical approach. It takes the output probability distributions (the softmax arrays) from all five standalone models and calculates the unweighted mathematical mean.
- **Intuition:** "Wisdom of the crowd." If the CNN is 99% confident something is a `food_poisoning` vector due to a specific phrase, but the ANN is only 40% confident, the voting mechanism smooths out the ANN's uncertainty.

3.3. Logistic Regression Stacking (Stacking LR)

- **Method:** An algorithmic meta-learner. The softmax probability outputs of the five base models are concatenated into a new feature vector (5 models × 3 classes = 15 features). A standard Scikit-Learn Logistic Regression model is trained on this 15-dimensional data.
- **Intuition:** Unlike soft-voting (which treats all models equally), the Logistic Regression model learns *which models to trust*. If the Transformer is highly accurate at identifying `poisonous` but terrible at `food_poisoning` , the Meta-Learner will mathematically learn to ignore the Transformer's advice when it predicts `food_poisoning` .

---

4. Overall Model Comparison & Performance

By evaluating the empirical performance across the architectures, distinct advantages and structural biases emerge:

1. **The Baseline (ANN):** Demonstrates high variance in accuracy between training runs (spanning 65% to 76%). Because it lacks spatial and sequential awareness, its decision boundaries are brittle and highly susceptible to the initial random states of the embedding matrix. It serves strictly as a fast embedder, not a standalone classifier.
2. **The Feature Extractor (CNN):** Achieves remarkably high precision on the majority classes (`edible` and `poisonous`). Because botanical descriptions often use highly rigid n-gram patterns (e.g., "green leafy vegetable", "deadly toxic mushroom"), the CNN easily maps these to hard classifications. However, it struggles with complex, nuanced narratives.
3. **The Global Context (Transformer):** Exhibits the most stable validation loss curve. Its multi-head attention excels at identifying long-range dependencies, preventing the model from being "tricked" by paragraphs that mention both edible foods and toxic outcomes in the same text.
4. **The Sequential Memories (LSTM & GRU):** These models provide the highest individual AUC scores. The addition of the custom `AdditiveAttention` layer allows for deep interpretability, actively

highlighting the specific medical or toxicological tokens driving the classification.

5. **The Unified Ensemble & Meta-Learners:** The ensembles consistently outperform all standalone models. By fusing the spatial mapping of the CNN with the sequential mapping of the RNNs, the stacking logistic regression successfully isolates the false positives generated by weaker branches, resulting in the highest weighted F1 score and the most robust ROC curves across all three classes.

---

5. Interpretation, Error Analysis & Reflection

Despite the high overall accuracy of the top-tier models, the confusion matrices and classification reports reveal a consistent, critical bottleneck: **The catastrophic failure to recall the `food_poisoning` class.**

5.1. The Root Cause of the Error

The models frequently produce False Negatives for `food_poisoning`, defaulting their predictions to `edible` or `poisonous`. This is driven by two intertwined factors:

1. **Severe Data Sparsity:** The original dataset contained only 152 samples of `food_poisoning` (representing roughly ~3.8% of the data). While `RandomOverSampler` corrects the class imbalance during the mathematical optimization step (preventing the gradient from ignoring the class entirely), it does not create *new* semantic data. The models are simply memorizing the same 152 paragraphs repeatedly.
2. **Ontological Lexical Overlap (The Vocabulary Problem):** `Edible` and `Poisonous` are intrinsic botanical states with distinct vocabularies ("harvest," "sweet" vs. "toxin," "fatal"). However, `food_poisoning` is a conditional state that occurs *to* edible items (e.g., "contaminated chicken," "unpasteurized milk"). Because the descriptions of food poisoning are overwhelmingly flooded with `edible` nouns, the models struggle to draw a clean mathematical hyperplane separating the two.

5.2. Strategic Reflection & Future Adjustments

Algorithmic interventions (such as tuning dropout rates, adjusting `class_weights`, or adding deeper attention layers) have reached a point of diminishing returns. The architecture is mathematically sound, but it is starved of semantic variety.

To resolve the `food_poisoning` recall failure, a strict **Data-Centric approach** is required. The minority class must be expanded with hundreds of new, unique paragraphs detailing foodborne pathogens, mycotoxins, parasites, and marine toxins. By scraping high-authority medical databases (CDC, WHO, FDA) for external data, the vocabulary matrix of the models will expand, providing the CNN filters and Transformer attention heads the specific, unique keywords required to isolate `food_poisoning` from standard `edible` botanicals.